Projektna naloga

PPJ

April 9, 2024

Cilj projektne naloge je izdelava domensko specifičnega jezika za opis infrastrukture v mestu. Za dokončanje projekta so potrebni naslednji koraki:

1. Zasnova jezika, določitev konstruktov in sintakse jezika
2. Priprava smiselnih testnih primerov
3. Definicija sintakse z gramatiko in regularnimi izrazi
4. Implementacija abstraktnega sintaktičnega drevesa
5. Konstrukcija končnega avtomata
6. Implementacija pregledovalnika (*scanner*)
7. Izračun FIRST in FOLLOW množic
8. Implementacija razčlenjevalnika (*parser*)
9. Implementacija pretvorbe v niz (*minifier* ali *pretty-printer*)
10. Implementacija izvoza v GeoJSON

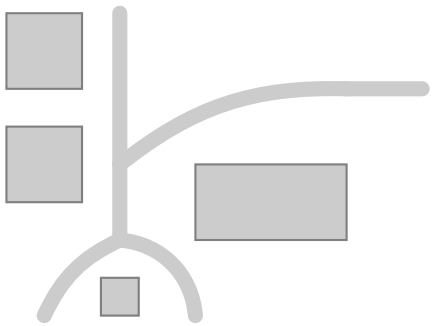
Jezik morate zasnovati sami, torej sami določite konstrukte in sintakso jezika.

Jezik opisan v tem dokumentu naj služi predvsem za inspiracijo.

1 Osnova

Zahtevano je, da lahko z uporabo vašega jezika opišete zgradbe in ceste v mestu. Torej je potrebno podpreti polilinije (*polyline*) in poligone (*polygon*). Na primer, majhen del mesta bi lahko opisali takole:

```
city "Fiesa" {
  road "Oljcna pot" {
    bend((1, 1), (2, 2), 20);
    bend((2, 2), (3, 1), 40);
    line((2, 2), (2, 5));
    bend((2, 3), (5, 4), 20);
    line((5, 4), (6, 4))
  };
  building "Soncne terase" {
    box((3, 2), (5, 3))
  };
  building "Dom Fiesa" {
    box((0.5, 3.5), (1.5, 2.5))
  };
  building "LaRocca" {
    box((0.5, 5), (1.5, 4))
  };
  building "Hotel Fiesa" {
    box((1.75, 1.5), (2.25, 1))
  }
}
```



1.1 Konstrukti

Uporabljen jezik vsebuje naslednje konstrukte.

1.1.1 Enota

Enota služi kot nevtralen element (NO-OP).

```
nil
```

1.1.2 Realna števila

```
1
1.2
```

1.1.3 Nizi

```
"NIZ"
```

1.1.4 Koordinate

S koordinatami lahko predstavimo lokacije na zemljevidu, v tem primeru je prva komponenta longituda, druga komponenta pa je latituda.

```
(X, Y)
```

1.1.5 Bloki

Blok `city` predstavlja mesto, ki ga opisujemo. Vsebuje lahko bloke za opis cest in zgradb. To je glaven element v jeziku.

```
city "NAME" {
  BLOCKS
}
```

Blok `road` predstavlja cesto. Vsebuje lahko ukaze za izris, ki izrišejo črte.

```
road "NAME" {
  COMMANDS
}
```

Blok `builing` predstavlja zgradbo. Vsebuje lahko ukaze za izris, ki izrišejo obrobo zapolnjenega lika. Obroba lika mora biti zaključena, torej končna lokacija mora biti enaka prvi.

```
building "NAME" {
  COMMANDS
}
```

1.1.6 Ukazi

Bloka `road` in `building` lahko vsebujeta naslednje ukaze za izris.

Ukaz `line` izriše črto, med podanima lokacijama.

```
line(POINT, POINT)
```

Ukaz `bend` izriše krivuljo, med podanima lokacijama. Podani kot določa stopnjo ukrivljenosti. Če je kot 0° se izriše ravna črta, če je kot 45° se izriše približek četrtine krožnice. Pozitivni koti pomenijo, da je krivulja nagnjena v levo, negativni koti pa pomenijo, da je krivulja nagnjena v desno.

```
bend(POINT, POINT, ANGLE)
```

Ukaz `box` izriše pravokotnik, med podanima lokacijama. Torej prva lokacija je zgornji levi kot pravokotnika, druga lokacija pa je spodnji desni kot.

```
box(POINT, POINT)
```

Ukaz `circ` izriše krog z izbranim polmerom na podani lokaciji. Ta lokacija predstavlja center kroga.

```
circ(POINT, RADIUS)
```

2 Nadgradnja

Jezik nadgradite s poljubnimi konstrukti in funkcionalnostmi, ki so smiselne za vaš projekt.

V nadaljevanju sledi nekaj idej.

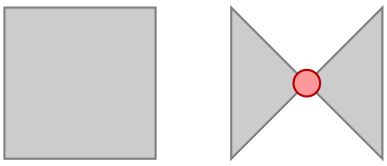
2.1 Validacija

Preverite smiselnost modela in uporabnika jezika opozorite na morebitne težave. Za pohitritev iskanja presečišč lahko uporabite R-drevo (dovoljena je uporaba knjižnice). Uporabite pa lahko tudi kateri koli drug temu namenjen algoritem iz računalniške geometrije.

2.1.1 Presečišča med poligoni

Z jezikom je mogoče opisati presekajoče poligone, prav tako pa lahko imajo poligoni presečišča tudi samim s sabo. V primeru, da ti predstavljajo zgradbe takšni poligoni niso smiselni. Prav tako bomo imeli težave pri izrisu. Zato lahko takšna presečišča najdemo in uporabnika nanj opozorimo.

```
city "Shapes" {
  building "Simple" {
    line((0, 0), (0, 2));
    line((0, 2), (2, 2));
    line((2, 2), (2, 0));
    line((2, 0), (0, 0))
  };
  building "Complex" {
    line((3, 0), (3, 2));
    line((3, 2), (5, 0));
    line((5, 0), (5, 2));
    line((5, 2), (3, 0))
  };
}
```



2.1.2 Presečišča med polilinijami

Pri polilinijah je manj težav, saj v primeru presečišč najverjetneje gre za nadvoz. Kljub temu lahko uporabimo podobno kodo kot za presečišča med poligoni in uporabnika nanj opozorimo.

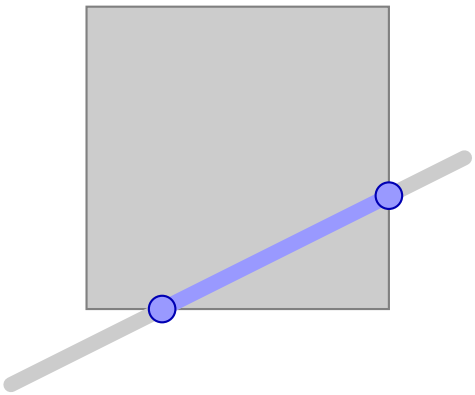
```
city "Shapes" {
  road "Simple" {
    line((0, 0), (0, 2));
    line((0, 2), (2, 2));
    line((2, 2), (2, 0));
    line((2, 0), (0, 0))
  };
  road "Complex" {
    line((3, 0), (3, 2));
    line((3, 2), (5, 0));
    line((5, 0), (5, 2));
    line((5, 2), (3, 0))
  };
}
```



2.1.3 Presečišča med poligoni in polilinijami

Prav tako lahko najdemo presešišča med poligoni in polilinijami. Najverjetne gre tukaj za napako. Sicer v redkih primerih se res lahko zgodi, da cesta gre skozi zgradbo, na primer pri bencinskih črpalkah.

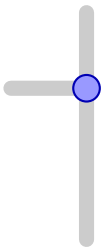
```
city "Shapes" {
  road "Intersects" {
    line((0, 0), (6, 3))
  };
  building "Building" {
    box((1, 1), (5, 5))
  }
}
```



2.1.4 Dotikališča med polilinijami

Dotikališča polilinij označujejo križišča. Uporabnika lahko opozorimo na njihove lokacije.

```
city "Junction" {
  road "Side road" {
    line((0, 2), (1, 2));
  };
  road "Main road" {
    line((1, 0), (1, 3));
  };
}
```



2.2 Elementi

Na zemljevid lahko dodamo ostale elemente mest, kot so reke, jezera, parke, polja. Mesta pa lahko vsebujejo tudi točkovne elemente, kot so drevesa, križišča, restavracije.

```
river "NAME" {
  COMMANDS
}

lake "NAME" {
  COMMANDS
}

park "NAME" {
  COMMANDS
}

field {
  COMMANDS
}

tree POINT

junction POINT

restaurant "NAME" POINT
```

2.3 Ukazi

Poleg osnovnih ukazov za izris lahko dodamo še druge: ukaz za izris elipse, dele krožnic, črte in krivulje sestavljene iz več segmentov (to je eden od načinov da se izognemo podvajanju točk), lahko definiramo razširjen ukaz `bend`, kjer se kota na vsaki strani razlikujeta, dodamo lahko parameter, ki nam pove kako ohlapna naj bo krivulja, lahko pa dodamo tudi ukaz `curve`, ki nam omogoča da prosto izberemo kontrolne točke Bezierjeve krivulje.

```
ellip(POINT, AXIS, AXIS)

arc(POINT, ANGLE, ANGLE)

polyline(POINTS)

polyspline(POINTS)
```

```
bend(POINT, POINT, ANGLE, ANGLE)

bend(POINT, POINT, ANGLE, ANGLE, LOOSENESS)

curve(POINT, POINT, CONTROL)

curve(POINT, POINT, CONTROL, CONTROL)
```

2.4 Sintaktični sladkor

Poleg (ali namesto) ukazov lahko uporabimo tudi bolj strnjeno in elegantno sintakso. Tukaj namesto ukaza `line` uporabljamo operator, prav tako pa združimo dele ceste, ki se držijo skupaj (tako se lahko izognemo podvajanju točk). Sintaktični sladkor lahko dodate kjerkoli se vam zdi smiselno.

```
road "Oljcna pot" {
  (1, 1) -- bend 20 (2, 2) -- bend 40 (3, 1);
  (2, 2) -- (2, 5);
  (2, 3) -- bend 20 (5, 4) -- (6, 4);
}
```

2.5 Konstrukti

2.5.1 Konstante in spremenljivke

Konstrukt `let` omogoča, da ustvarite novo konstanto. Tako boste lahko določene točke shranili in jih nato večkrat uporabili. Predlagamo, da uporabite dinamično in strogo tipiziranje. Po želji lahko dodate tudi spremenljivke.

```
road "Oljcna pot" {
  let p = (2, 2);
  let q = (5, 4);

  bend((1, 1), p, 20);
  bend(p, (3, 1), 40);
  line(p, (2, 5));
  bend((2, 3), q, 20);
  line(q, (6, 4))
}
```

2.5.2 Matematični izrazi

Pogosto je koristno, da lahko izračunamo novo lokacijo na podlagi parametrov ali obstoječih lokacij. Da boste lahko dobili elemente točk je potrebno vgraditi funkciji `fst` in `snd`, ki vrneta prvi in drugi element točke.

```
road "Oljcna pot" {
  let bound = 1;
  let s = (bound, bound);
  let p = (fst(s) + 1, snd(s) + 1);
  let q = (5, 4);

  bend(s, p, 20);
  bend(p, (3, bound), 40);
  line(p, (2, 5));
  bend((2, 3), q, 20);
  line(q, (6, 4))
}
```

2.5.3 Kontrola toka

```
building "Hotel Fiesa" {
  if n > 1 {
    line((1, 1), (2, 2))
  } else {
    line((2, 1), (1, 2))
  }

  for i = 1 to 3 {
    box((1, i), (2, i + 1))
  }
}
```

2.5.4 Abstrakcija

Določene kompleksne elemente lahko ponovimo na poljubnih mestih, tako da dodamo procedure **`procedure`** in funkcije **`lambda`**. Poleg tega, pa lahko uporabimo ta konstrukta za poljubne izračune, kar še dodatno dvigne izraznost jezika. Za rekurzivne funkcije lahko dodamo konstrukt **`rec let`**. Če so spremenljivke dinamično tipizirane pa lahko rekurzijo podpremo tudi brez dodatnih konstruktov, tako da definiramo operator, ki nam vrne fiksno točko `fix` [1]. Predstavljene proce-

[1] https://en.wikipedia.org/wiki/Fixed-point_combinator

dure in funkcije vrnejo zadnjo vrednost, lahko pa definiramo tudi ukaz `return`, ki ga lahko uporabimo tudi za predčasen zaključek izvajanja.

```
procedure row(p) {
  for i = 1 to 3 {
    box((fst(p), snd(p) + i), (fst(p) + 1, snd(p) + i + 1))
  }
};

building "Hotel Fiesa" {
  let square = lambda(x) {x * x};
  row((1, square(2)))
}
```

2.5.5 Seznami

Koordinate lahko služijo tudi kot pari. Z uporabo parov je mogoče enostavno podpreti enojno povezane sezname. Za rep para lahko uporabimo enoto `nil`. Podpremo lahko tudi zanko `foreach`, ki se izvede za vsak element seznama. Prav tako pa lahko definiramo tudi lepšo sintakso, ki se pretvori v predstavitev s pari.

```
building "Hotel Fiesa" {
  let list = (1, (2, (3, nil)));

  foreach x in list {
    box((1, x), (2, x + 1))
  };

  let fancy = [1, 2, 3]
}
```

2.5.6 Operacije nad poligoni

Implementiramo lahko operacije nad poligoni, unijo, presek, razliko. Za implementacijo razike moramo podpreti tudi poligone z luknjami.

2.5.7 Tansformacije

Dodamo lahko tudi transformacije referenčnega okvirja. Na primer, lahko premaknemo izhodišče in nato lokacije sledečih konstruktov opišemo relativno glede na premaknjeno izhodišče.

```
city "Positions" {
  translate (5, 3);
  marker (0, 0)
}
```

2.6 3D

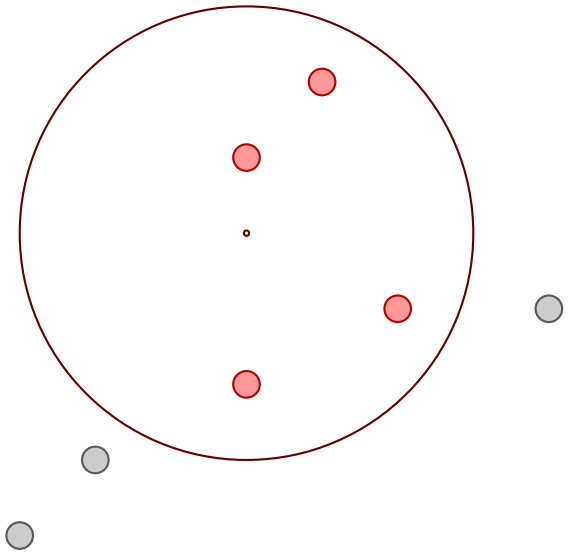
Poleg longitude in latitude lahko dodamo točkam še nadmorsko višino ali višino stavbe. V primeru, da boste izdelali 3D izrisovalnik lahko za izris uporabite tudi te informacije.

2.7 Povpraševanja

Dodate lahko povpraševanja nad modelom, torej lahko poiščemo elemente znotraj regije zanimanja. Na primer, če imamo podano trenutno lokacijo lahko poiščemo restavracije, ki so v bližini. Povpraševanja so lahko del jezika za opis infrastrukture ali samostojen jezik. Tudi tukaj lahko za pohitritev iskanja uporabimo R-drevo.

```
city "Restaurants" {
  marker (0, 0);
  marker (1, 1);
  marker (3, 2);
  marker (7, 3);
  marker (4, 6);
  marker (5, 3);
  marker (3, 5);

  let roi = neigh((3, 4), 3);
  foreach x in roi {
    highlight x
  }
}
```



2.8 Meta podatki

Vsakemu elementu v mestu lahko dodamo poljubne meta podatke. Te metapodatke lahko kasneje uporabimo tudi za obogatitev izrisa.

```
city "Monitoring stations" {
  marker "Edvard Rusjan" (8, 3) {
    set("precipitation", 0.93);
  }
}
```

2.9 Povezani podatki

Metapodatke lahko pridobimo direktno iz zunanjih virov. Na primer, tako podatke o infrastrukturi direktno povežemo z podatki iz merilnih postaj.

```
let data = link("https://www.monitoring-station.com/json");

city "Monitoring stations" {
  marker "Edvard Rusjan" (8, 3) {
    set("precipitation", data["Edvard Rusjan"].precipitation)
  }
}
```

2.10 Časovno odvisni podatki

Elemenom lahko dodamo tudi čas nastanka, tako lahko opišemo spremembe v infrasktrukturi skozi neko časovno obdobje.

2.11 Dostop do stanja izrisovalnika

Jeziku lahko dodamo dostop do stanja izrisovalnika. Dodamo lahko pretekel čas, kar nam omogoča opis animacij. Dodamo lahko trenutno pozicjo miške, torej lahko določimo regijo zanimanja z miško. Dodamo lahko vgrajene funkcije, torej lahko znotraj jezika izvedemo kompleksne operacije, ki so implementirane v sklopu izrisovalnika.